EC8208 – Software Architecture

Continuous Assessment 01

Architecture Analysis and Design Report

Project:

University Volunteering & Event Management Web App

Group Members:

EG/2021/4598 – Karunarathna C.K

EG/2021/4465 – Silva K.G.B.P

EG/2021/4392 – Akarshana A.N

EG/2021/4632 – Kumarasinghe K.K.R

EG/2021/4580 – Jayasri M.S.P.D.R.

EG/2021/4524 – Gunarathna B.V.I.D.W

13th July 2026

Table of Contents

1. Introduction

1.1 Background of the Selected Problem

Universities are home to a vibrant ecosystem of clubs, societies, and student organizations that regularly organize volunteering events, community service drives, and extracurricular activities. These events play a vital role in student development, fostering leadership skills, community engagement, and personal growth. However, the process of managing these activities remains largely manual and fragmented.

Currently, most campus clubs collect volunteer sign-ups manually through tools such as Google Forms or WhatsApp groups. There is no single, centralized dashboard for students to view all open volunteering opportunities or track work and also event organizers struggle to find volunteers for events. This leads to missed opportunities, duplicated effort, inconsistent record-keeping, and a lack of transparency for both students and organizers.

1.2 Problem Statement

The absence of a centralized digital platform for university volunteering creates several problems:

- Students have no unified way to discover volunteering opportunities across all campus clubs.
- Club organizers spend excessive time on administrative tasks specially for finding volunteers.
- There is no structured mechanism for defining volunteer roles per event, managing applications, or communicating decisions to volunteers in a timely manner.
- University administration lacks visibility into student volunteer engagement data.

These issues highlight the need for a centralized web application that automates event management, volunteer role definition, and application workflow for the entire campus community.

1.3 Objectives of the Proposed System

The primary objectives of the proposed system are:

- To provide a unified web portal where campus clubs can publish and manage volunteering events.
- To Enable Organizers to find relevant volunteers for event.
- To enable students to search, discover, and apply for volunteering opportunities from a single platform.
- To provide volunteers with a personal dashboard displaying their event application history and statuses.
- To implement role-based access control distinguishing between Volunteers and Super Administrators, with event-scoped Organizer permissions.

2. Requirements Analysis

2.1 Functional Requirements

The following functional requirements have been identified for the system:

Req. ID	Requirement	Description
FR-01	User Registration & Login	Volunteer must be able to register with an email, create a password, and log in securely. The system shall support role assignment (Volunteer, Super Admin), with Organizer permission granted per event by the Super Admin.
FR-02	Role-Based Access Control	The system shall restrict functionality based on user roles. Volunteers can browse events and apply for roles; Organizers can manage roles and applications for their assigned event; Super Admins have full system access.
FR-03	Event Creation & Management	The Super Admin must be able to create, edit, publish, and delete volunteering events with details including title, description, date, location, and status. The Super Admin can assign a Volunteer as the Organizer for a specific event.
FR-04	Event Discovery & Search	Volunteers must be able to browse and view upcoming volunteering events and available roles with slot counts.
FR-05	Event Registration	Volunteers must be able to apply for a specific role within an event. The system shall enforce slot capacity limits per role and prevent duplicate applications.
FR-06	Attendance Tracking	The Organizer must be able to define, update, and remove volunteer roles (role name, available slots) for their assigned event.
FR-07	Hours Verification & Dashboard	The system shall calculate and display verified applications for their event, and approve or reject individual applications per volunteer role.
FR-08	Notifications	The system shall send email notifications to volunteers when their application status changes to Approved or Rejected.

2.2 Non-Functional Requirements

Req. ID	Attribute	Description
NFR-01	Performance	The event listing page shall load within 2 seconds under normal network conditions. API response times shall not exceed 500ms for standard queries.
NFR-02	Scalability	The system shall support at least 1,000 concurrent users during peak event registration periods without performance degradation.
NFR-03	Security	User identity and credentials are fully managed by Azure AD B2C, eliminating plaintext password storage. API endpoints are protected with JWT-based authentication validated by Azure API Management before requests reach the backend. HTTPS is enforced at both the Azure Static Web Apps and API Management layers.
NFR-04	Usability	The user interface shall be responsive (mobile and desktop) and intuitive enough that a new student can register for an event without any training.
NFR-05	Availability	The system shall target 99.5% uptime, achieved through cloud-based hosting with automated health checks and restarts.
NFR-06	Maintainability	The codebase shall follow modular design principles with clear separation between layers, enabling independent updates to the UI, API, and database without cross-layer side effects.

2.3 Stakeholder Identification

Stakeholder	Role	Interest / Concern
Volunteers	Primary End Users	Discover events, apply for volunteer roles, and receive timely application status updates.
Event Organizers	Event Managers	Manage volunteer roles and applications for their assigned event efficiently.
Administrator	Oversight / Governance	Access aggregated data on student volunteering engagement for institutional reporting and accreditation.

3. Architecture Selection

3.1 Identified Architecture Styles

Before selecting the final architecture, the following common architecture styles were evaluated for their suitability to the problem domain:

Architecture Style	Pros	Cons
Monolithic	Simple to develop and deploy initially.	Difficult to scale and maintain as the application grows; tight coupling between components.
3-Tier / Layered	Clear separation of concerns; easy to understand and develop; each layer can be developed and tested independently; maps directly to web app structure.	Can introduce latency between layers; strict layering may be too rigid for some use cases.
Microservices	Highly scalable; each service can be deployed independently; technology flexibility per service.	Significant operational overhead; complex inter-service communication; overkill for a prototype or small-to-medium application.
Event-Driven	Excellent for real-time features and loose coupling between producers and consumers.	Increased complexity in debugging and tracing event flows; not necessary when most interactions are synchronous request-response.

3.2 Selected Architecture and Justification

Selected Architecture Style: 3-Tier / Layered Architecture

The 3-T **The 3-Tier Layered Architecture** was selected as the most suitable architectural style for this project because the system is designed for a university environment with a relatively fixed number of users and predictable request volumes. The application does not require the high scalability or asynchronous processing provided by event-driven architectures, as most operations are straightforward request-response interactions such as event management, volunteer registration, and application approval. Similarly, a microservices architecture was not adopted because the system consists of a single cohesive business domain with tightly related

functionalities. Splitting the application into multiple independent services would introduce unnecessary complexity in deployment, communication, monitoring, and maintenance. Instead, a 3-tier layered architecture provides clear separation between the presentation, business logic, and data layers while remaining simple, maintainable, secure, and well-suited to the project's functional and non-functional requirements.

- **Presentation Layer:** Frontend UI built with React, responsible for rendering the user interface and handling user interactions.
- **Business Logic Layer:** A REST API backend built with Node.js and Express, responsible for processing application rules, authentication, event management.
- **Data Storage Layer:** A database layer using Azure Database for PostgreSQL responsible for persisting all application data including user accounts, events, event roles and applications.

Justification:

- Layered architecture provides a clear separation of concerns, making the system easier to develop, test, and maintain.
- It directly maps to the CA3 prototype requirements (UI, Business Logic, Data Storage).
- It enables straightforward task division among team members (frontend, backend, database).
- It is the industry standard for web applications of this scope and complexity.

4. Architectural Design

This section presents the architectural design of the system through multiple diagram views. Each diagram provides a different perspective on the system's structure and behavior.

4.1 Context Diagram

The Context Diagram provides a high-level view of the system and its interactions with external actors. The Campus Volunteering & Event Management System is the central entity, interacting with three primary actors: Students (who search and apply for events), Organizers (who manage assigned events, and manage and choose volunteer applications), and Super Administrators (who create events and check overall system). An external Email Service is also shown for sending notifications.

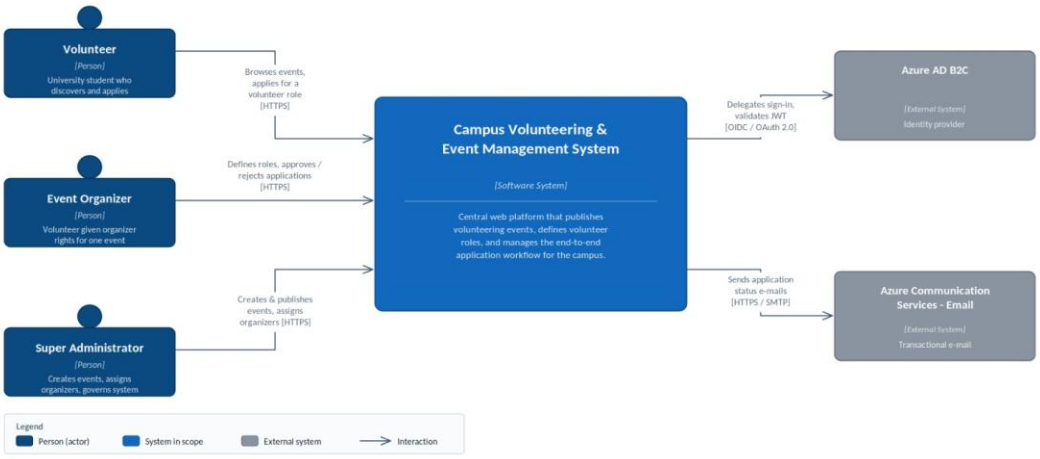

Figure 1: Context Diagram

4.2 Component Diagram

The Component Diagram shows the internal structure of the system, decomposed into its major components across the three architectural layers. The Presentation Layer contains the Volunteer Student Portal and Admin Portal components. The Business Logic Layer Event Management, User Management, Application Management and Event Roles Management. As External Services there are Notification service (Azure Email Service) and Authentication Service (Azure AD B2C). The Data Storage Layer consists of the Users, Events, Applications, and Event Roles collections in PostgreSQL.

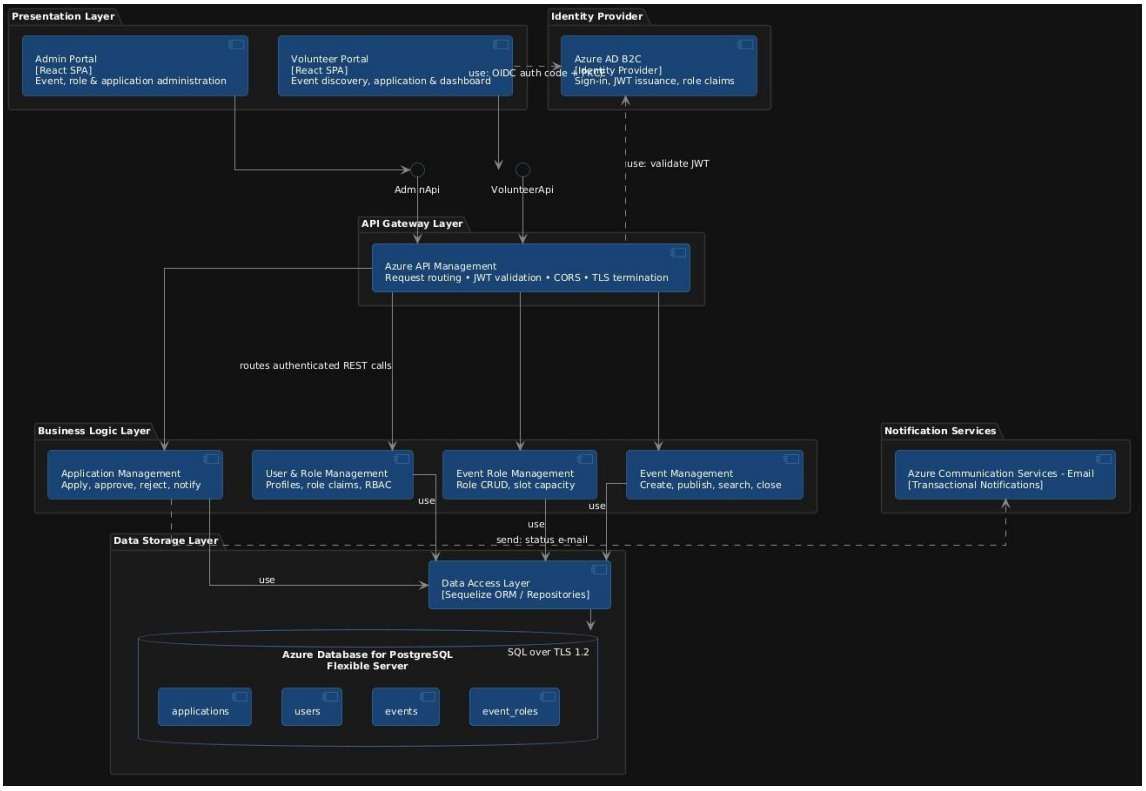

Figure 2: Component Diagram

4.3 Deployment Diagram

The deployment architecture hosts the Admin Portal and Volunteer Portal on Azure Static Web Apps with GitHub Actions providing automatic CI/CD. User authentication is managed by Azure AD B2C, which issues JWT tokens containing user role claims. All client requests are routed through Azure API Management, which validates tokens, enforces security policies such as rate limiting, and securely forwards requests to the backend through Virtual Network (VNet) integration.

The backend application runs as a containerized service on Azure Container Apps with internal-only access inside a private VNet. It communicates securely with Azure Database for PostgreSQL through VNet injection, while Azure Container Registry, Azure Key Vault, and Azure Communication Services are accessed via private endpoints. Managed Identity enables the backend to retrieve secrets from Key Vault without storing credentials in code. Azure RBAC controls infrastructure access, and all communication is protected using HTTPS/TLS. As a result, all internal service communication remains within the private network, with the only public communication being the final delivery of email notifications from Azure Communication Services to users.

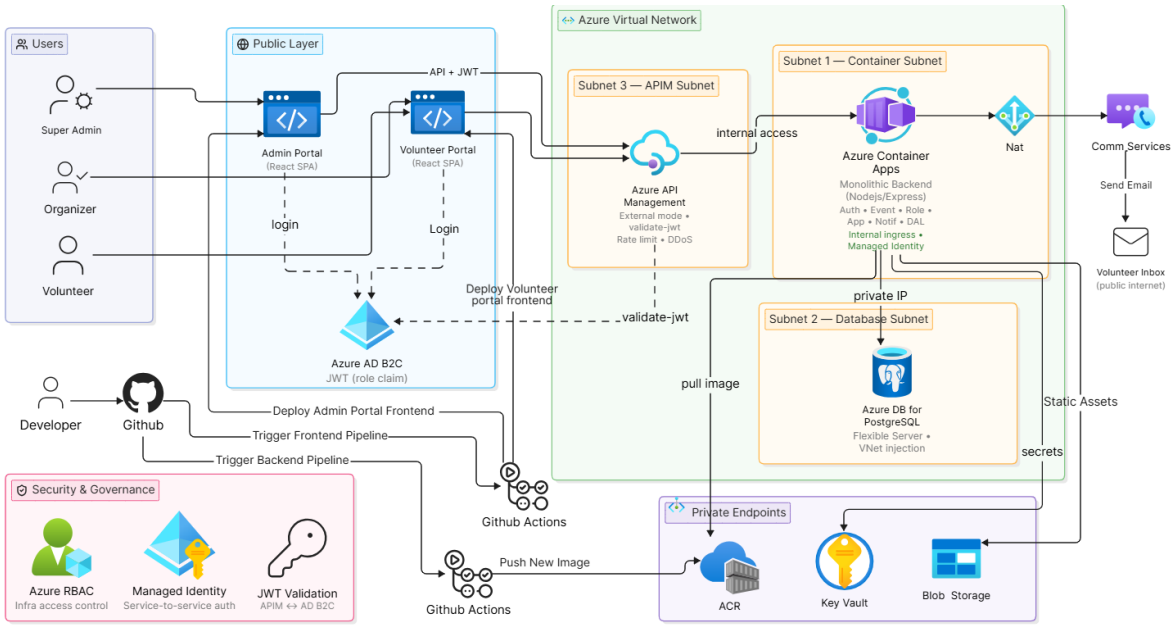

Figure 3: Deployment Diagram

4.4 Class Diagram

The Class Diagram represents the static structure of the system's core domain model. The primary entities are User, Event, EventRole, and Application. A User can hold one of two system-level roles: Volunteer or Super Admin. An Event is created by a Super Admin and has one assigned Organizer (who is also a Volunteer in the system). Each Event contains multiple EventRoles, where each role defines a name and the number of available slots. Volunteers submit Applications targeting a specific EventRole, and each Application carries a status of Pending, Approved, or Rejected managed by the Organizer of that event.

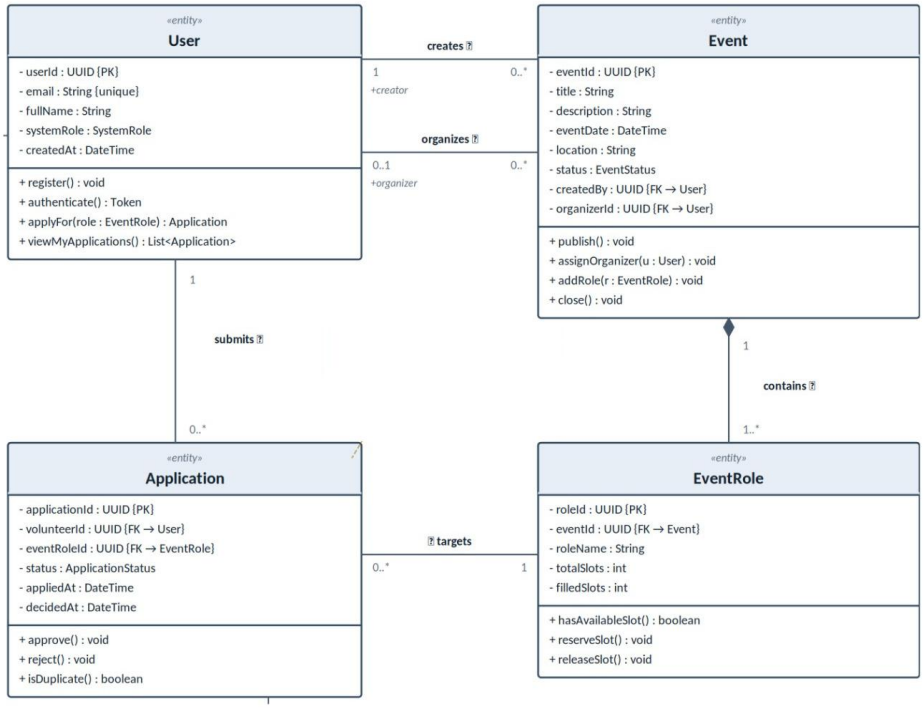

Figure 4: Class Diagram

4.5 Data Flow Diagram

The Data Flow Diagram illustrates the movement of data between users, system processes, data stores, and external services within the volunteer management system. Super Admins publish events and assign organizers, with event and user information stored in the database. Volunteers submit applications for event roles, which are recorded and validated against available positions. Event Organizers review applications, manage volunteer roles, and approve or reject requests. Based on the decision, the system sends email notifications to volunteers through the Azure Email Service. The diagram highlights how data flows between the core processes while maintaining centralized storage and communication.

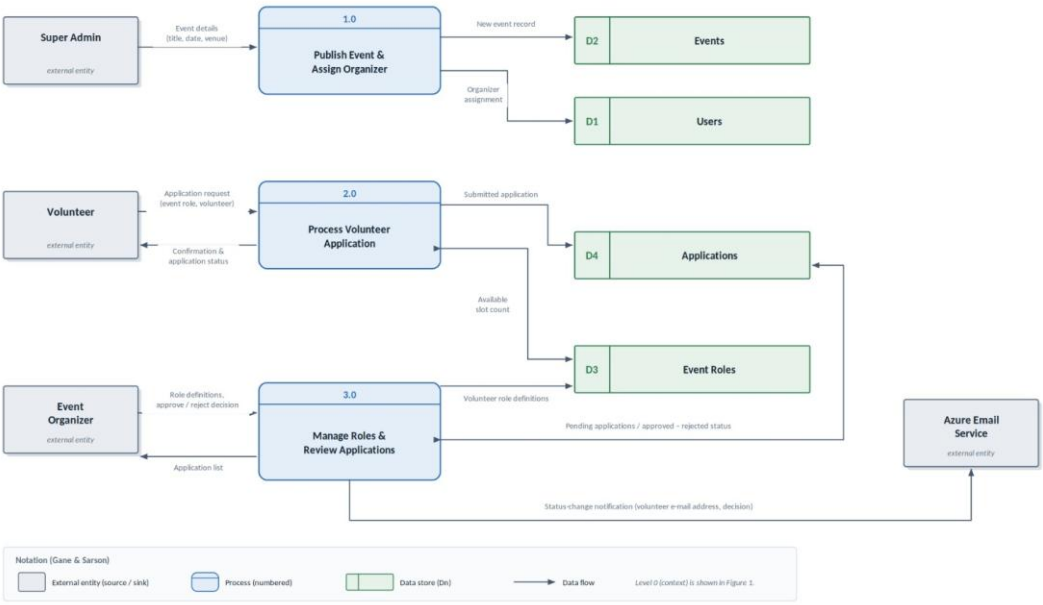

Figure 5: Data Flow Diagram

5. Technology Stack Selection

The technology stack has been selected to align with the 3-Tier Layered Architecture and to ensure rapid prototyping capability for the CA3 deliverable.

Layer / Area	Technology	Justification
Presentation Layer	React	Component-based UI framework enabling fast, interactive single-page applications.
Business Logic Layer	Node.js with Express.js	Lightweight, high-perform JavaScript runtime for building RESTful APIs with a vast npm ecosystem.
Data Storage Layer	Azure Database for PostgreSQL Flexible Server	Managed relational PostgreSQL database with native foreign key support critical to the system data model. VNet private access, high availability standby replica automated backups..
Infrastructure	Microsoft Azure	Azure services offering comprehensive cloud infrastructure with built-in CI/CD support via GitHub Actions
Authentication	Azure AD B2C	Managed identity provider handling JWT issuance, token refresh, user flows, and custom role claims (system_role: SUPER_ADMIN / VOLUNTEER). Eliminates custom auth implementation in the backend entirely.
API Gateway	Azure API Management	Handles REST request routing, JWT validation via Azure AD B2C policy, and rate limiting before requests reach the backend container.
Version Control and CICD	Git + GitHub + GitHub Actions	Industry-standard version control and Integrated CICD.
Frontend Deployment	Azure Static Web Apps (Admin & Volunteer Portals)	Automatically deploy Static web apps to Azure from a code repository.
Backend Deployment	Azure Container Apps	Inbuilt Serverless Auto-scaling on Container Applications.
Notifications	Azure Communication Service	highly scalable, secure, production-ready email infra for applications.
Secrets Management	Azure Key Vault	securely stores sensitive secrets.

6. Quality Attributes

This section discusses how the proposed architecture addresses key quality attributes.

6.1 Scalability

The 3-Tier architecture allows each layer to scale independently. The frontend, being a static single-page application, can be served through a CDN for global scalability. The backend API server can be horizontally scaled by deploying multiple instances behind a load balancer. The database layer scales through Azure Database for PostgreSQL Flexible Server replica instances for read distribution and storage auto-scaling as data volume grows. The Azure-based deployment strategy (Azure Static Web Apps, Azure Container Apps, Azure Database for PostgreSQL Flexible Server) provides managed infrastructure with auto-scaling capabilities that dynamically adjust to demand.

6.2 Performance

React ensures a responsive single-page application experience by minimizing full-page reloads and using virtual DOM diffing for efficient UI updates. On the backend, Express middleware enables efficient request processing. Database queries will be optimized through proper indexing on frequently queried fields (event date, user ID, registration status). Response caching will be implemented for frequently accessed data.

6.3 Security

Security is implemented across all layers:

- **Transport Security:** All client-server communication uses HTTPS encryption.
- **Authentication:** JWT tokens are used for stateless authentication. Token role claims for authorization. Token expiry and refresh mechanisms prevent unauthorized access.
- **DDoS Protection & Rate Limiting:** Azure API Management provides built-in rate limiting and throttling policies, preventing denial-of-service attacks before they reach the backend container. This protects both the API and DB.
- **Cloud IAM:** Azure Role-Based Access Control (Azure RBAC) governs access to all Azure resources at secure. Managed Identities are used for service-to-service communication eliminating hardcoded credentials anywhere in the codebase.

- **Secrets Management:** Azure Key Vault securely stores sensitive secrets, including the PostgreSQL DB. Azure Container Apps retrieves these secrets at runtime using a Managed Identity. No hardcoded credentials.

6.4 Availability

The system is deployed on Azure (Static Web Apps, Container Apps, Managed PostgreSQL Flexible Server) which provides high-availability across Availability Zones with automatic health checks, container restarts, and database failover mechanisms. Azure Database for PostgreSQL in high availability mode provides automatic standby failover with typical recovery times under two minutes. The target availability is 99.5%. The stateless nature of the backend (JWT-based auth, no server-side sessions) means any instance can handle any request, enabling seamless failover between instances.

6.5 Maintainability

The layered architecture enforces a strict separation. Changes to the user Presentation Layer do not affect the Business Logic Layer or database (Data Storage Layer), Each layer has well-defined interfaces (REST APIs), making it possible to modify or replace any layer without impacting others. The modular component-based approach in React and the controller-service pattern in Express further enhance maintainability.

6.6 Reliability

The database employs automated backups (continuous backups and point-in-time recovery on Azure Database for PostgreSQL Flexible Server) to prevent data loss. Critical operations such as event registration are wrapped in database transactions to ensure atomicity - either the registration succeeds completely, or it rolls back. Comprehensive error handling is implemented across all API endpoints, returning meaningful error messages to the client and logging detailed error information server-side for debugging. Input validation at multiple layers (client, API middleware, database constraints) ensures data integrity.

7. Conclusion

This report has presented a comprehensive architecture analysis and design for the 'Campus Volunteering & Event Management Web App.' The proposed system addresses the real-world problem of fragmented volunteer management across university campus clubs by providing a centralized, automated platform.

The 3-Tier Layered Architecture was selected as the most suitable architectural style after evaluating multiple alternatives. This architecture provides clear separation of concerns across the Presentation Layer (React on Azure Static Web Apps), Business Logic Layer (Node.js/Express on Azure Container Apps), and Data Storage Layer (Azure Database for PostgreSQL), enabling independent development, testing, and scaling of each layer.

The system's three major functionalities – User & Role Management, Event Publishing & Volunteer Role Definition, and Application Management & Notifications – have been thoroughly analyzed through functional and non-functional requirements. The architectural design has been illustrated through five diagram types: Context, Component, Deployment, Class, Data Flow diagrams.

The selected technology stack and quality attribute analysis demonstrate that the proposed solution is scalable, secure, maintainable, and reliable. The Azure-based deployment strategy ensures the system meets its scalability, availability, and security quality attributes while remaining straightforward to implement as a functional prototype in the CA3 deliverable.